



Getting Started With LangChain

Installation

Before installation you need to generate an openai API key and use it in LangChain applications

Install the LangChain Libraries

- pip install langchain langchain-openai langchain-community
- pip install langchain-text-splitters langchain-postgres
- :

LLMs are the Driving Engines

- LLMs are the driving engine behind most generative AI applications. LangChain provides two simple interfaces to interact with any LLM API provider:

Chat models

LLMs

- The LLM interface simply takes a string prompt as input, sends the input to the model provider, and then returns the model prediction as output.
- Let's import LangChain's OpenAI LLM wrapper to invoke a model prediction using a simple prompt:

Using LLM

```
from langchain_openai.llms import OpenAI  
model = OpenAI(model="gpt-3.5-turbo")  
model.invoke("The sky is")
```

Parameters to Configure

Other useful parameters to configure include the following, offered by most providers.

1- temperature

This controls the sampling algorithm used to generate output. Lower values produce more predictable outputs (for example, 0.1), while higher values generate more creative, or unexpected, results (such as 0.9). Different tasks will need different values for this parameter. For instance, producing structured output usually benefits from a lower temperature, whereas creative writing tasks do better with a higher value:

2- max_tokens

This limits the size (and cost) of the output. A lower value may cause the LLM to stop generating the output before getting to a natural end, so it may appear to have been truncated.

The Chat Model Interface

```
from langchain_openai.chat_models import ChatOpenAI
from langchain_core.messages import HumanMessage

model = ChatOpenAI()
prompt = [HumanMessage("What is the capital of France?")]

model.invoke(prompt)
```

Types of Messages

Instead of a single prompt string, chat models make use of different types of chat message interfaces associated with each role mentioned previously. These include the following:

- *HumanMessage*

A message sent from the perspective of the human, with the user role

- *AIMessage*

A message sent from the perspective of the AI that the human is interacting with, with the assistant role

- *SystemMessage*

A message setting the instructions the AI should follow, with the system role

- *ChatMessage*

A message allowing for arbitrary setting of role

- Let's incorporate a *SystemMessage* instruction in our example:

Simple Example

```
from langchain_core.messages import HumanMessage, SystemMessage
from langchain_openai.chat_models import ChatOpenAI
```

```
model = ChatOpenAI()
system_msg = SystemMessage(
    "You are a helpful assistant that responds to questions with three
    exclamation marks."
)
human_msg = HumanMessage('What is the capital of France?')
```

Prompts

```
from langchain_core.prompts import PromptTemplate  
template = PromptTemplate.from_template("""Answer the question based on the  
context below. If the question cannot be answered using the information provided, answer with "I  
don't know".
```

```
Context: {context}
```

```
Question: {question}
```

```
Answer: """)
```

```
template.invoke({
```

```
    "context": """The most recent advancements in NLP are being driven by Large  
Language Models (LLMs). These models outperform their smaller  
counterparts and have become invaluable for developers who are creating  
applications with NLP capabilities. Developers can tap into these  
models through Hugging Face's `transformers` library, or by utilizing  
OpenAI and Cohere's offerings through the `openai` and `cohere`  
libraries, respectively.""",
```

```
    "question": "Which model providers offer LLMs?"
```

```
})
```

Building a Chat Application?

- If you're looking to build an AI chat application, the ChatPromptTemplate can be used instead to provide dynamic inputs based on the role of the chat message:

ChatPromptTemplate

```
from langchain_core.prompts import ChatPromptTemplate

template = ChatPromptTemplate.from_messages([
    ('system', "Answer the question based on the context below. If the
        question cannot be answered using the information provided, answer with
        'I don't know.'"),
    ('human', 'Context: {context}'),
    ('human', 'Question: {question}'),
])

template.invoke({
    "context": "The most recent advancements in NLP are being driven by Large
        Language Models (LLMs). These models outperform their smaller
        counterparts and have become invaluable for developers who are creating
        applications with NLP capabilities. Developers can tap into these
        models through Hugging Face's `transformers` library, or by utilizing
        OpenAI and Cohere's offerings through the `openai` and `cohere`
        libraries, respectively.",
    "question": "Which model providers offer LLMs?"
})
```

A Complete Chatbot

```
import {ChatOpenAI} from '@langchain/openai'
import {ChatPromptTemplate} from '@langchain/core/prompts'
import {RunnableLambda} from '@langchain/core/runnables'
// the building blocks
const template = ChatPromptTemplate.fromMessages([
  ['system', 'You are a helpful assistant.'],
  ['human', '{question}'],
])
const model = new ChatOpenAI()
// combine them in a function
// RunnableLambda adds the same Runnable interface for any function you write

const chatbot = RunnableLambda.from(async values => {
  const prompt = await template.invoke(values)
  return await model.invoke(prompt)
})
// use it
await chatbot.invoke({
```

LangGraph

LangGraph is a library built on top of LangChain, designed to add cyclic computational capabilities to your LLM applications.

While LangChain allows you to define chains of computation (Directed Acyclic Graphs or DAGs), LangGraph introduces the ability to add cycles, enabling more complex, agent-like behaviors where you can call an LLM in a loop, asking it what action to take next.

LangGraph

- LangGraph is a powerful tool for building stateful, multi-actor applications with Large Language Models (LLMs).
- It extends the LangChain library, allowing you to coordinate multiple chains (or actors) across multiple steps of computation in a cyclic manner.

Langraph Key Concepts

- **Stateful Graph:** LangGraph revolves around the concept of a stateful graph, where each node in the graph represents a step in your computation, and the graph maintains a state that is passed around and updated as the computation progresses.
- **Nodes:** Nodes are the building blocks of your LangGraph. Each node represents a function or a computation step. You define nodes to perform specific tasks, such as processing input, making decisions, or interacting with external APIs.
- **Edges:** Edges connect the nodes in your graph, defining the flow of computation. LangGraph supports conditional edges, allowing you to dynamically determine th

A Very Simple Example

```
from langgraph.graph import StateGraph, START, END
# Define agents
def research_node(state):
    # Research logic here
    return {"messages": [{"content": "Research complete", "name": "researcher"}]}

def chart_node(state):
    # Chart generation logic here
    return {"messages": [{"content": "Chart generated", "name": "chart_generator"}]}

# Build multi-agent graph
workflow = StateGraph()
workflow.add_node("researcher", research_node)
workflow.add_node("chart_generator", chart_node)
workflow.add_conditional_edges("researcher", lambda state: "chart_generator",
{"chart_generator": "chart_generator"})
workflow.add_edge(START, "researcher")
workflow.add_edge("chart_generator", END)
```

Building an Agent Using LanGraph

- `pip install duckduckgo-search`